这一讲，管住越来越长的上下文



1 上下文涨得多快，为什么一长就又慢、又贵、还忘事?

2 模型层和服务层，怎么让 1M token 读得了、也读得起?

3 真塞不下了，agent 怎么压缩历史?

上一讲的问题：摘要 / 检索 / 子 agent，各丢什么 → 在问题 3 里回答 子 agent 后面专门讲

历史线性增长，总输入按平方增长

第 t 次的输入 = 固定前缀 + 这次之前的全部历史



一直涨下去

固定前缀

之前的历史

累计输入

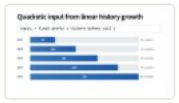
前缀按第 1 次的 6K 画（示意）

每次输入、累计：课件 p.3

编码 agent 会话

连着用一整天、

甚至好几天



课件 p.2–3（中文重绘）

一个 agent 的提示词里装了什么

77,922 平均每个会话的 token · 1,500 个 OpenHands 会话

还只是单个任务的短会话

系统提示词：编码 agent 一般 8K–20K　OpenHands 15K–18K ↓



↑ 直接回复

思考 ≈ 回复的 5 倍

读一个文件　跑一条命令

每一步都带着这一整套 → 加起来涨得非常快

"直接回复"、8K–20K、5 倍：课上口头补充



课件 p.4（中文重绘）

上下文一长：两个挑战

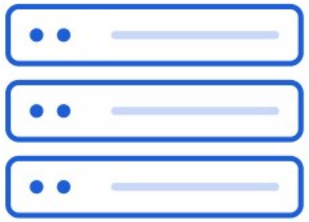
能力 Capacity



模型能不能用上它需要的那条证据?

架构 · 位置编码 · 训练
→ 评测把真正能用的长度量出来

效率 Efficiency



系统服务得起吗?

注意力 · KV 显存 · 缓存 · 压缩
→ 成本和延迟



课件 p.5（中文重绘）

效率背后，是非常大的工程

Claude Code · Codex：用着理所当然



2,500 万

据说是 Codex 的日活

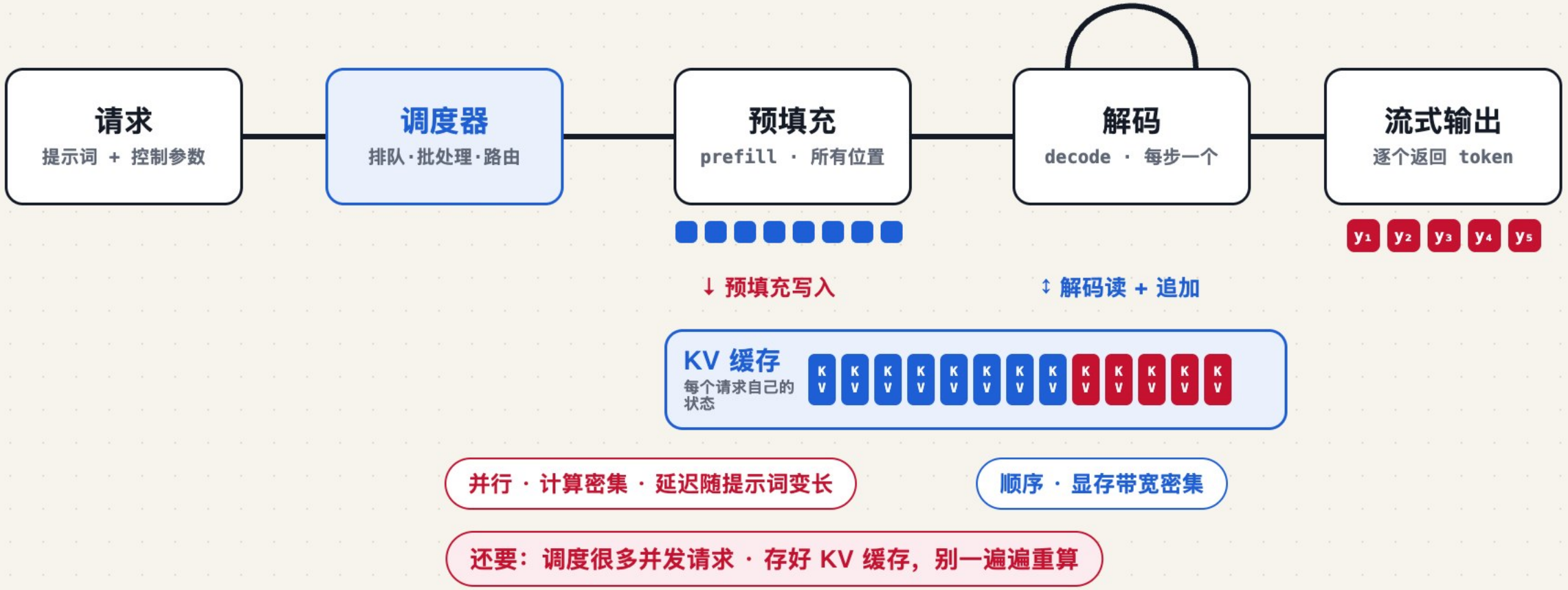
每人每隔几秒请求一次

据课堂所述，未核实



课件 p.5（中文重绘）

一次请求在推理服务里怎么走





课件 p.6–8（中文重绘）

衡量服务的四个指标

TTFT

首 token 时间



从请求到达，到看到第一个输出 token

TPOT

输出 token 的间隔



字与字之间隔多久

吞吐

整个系统的流量



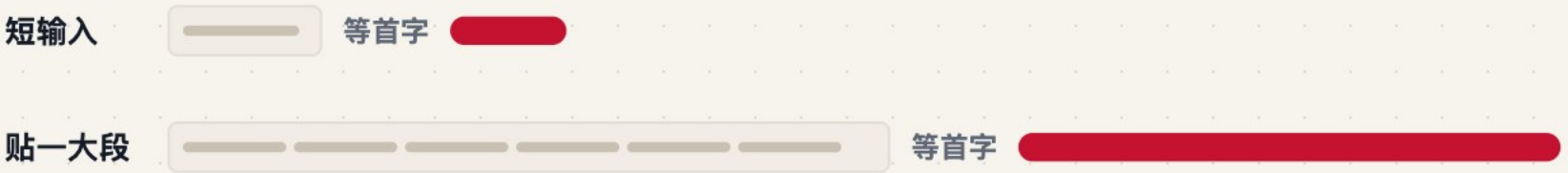
单位时间完成多少 token 或请求

成本

三部分 token 费用



输入 + 缓存输入 + 输出



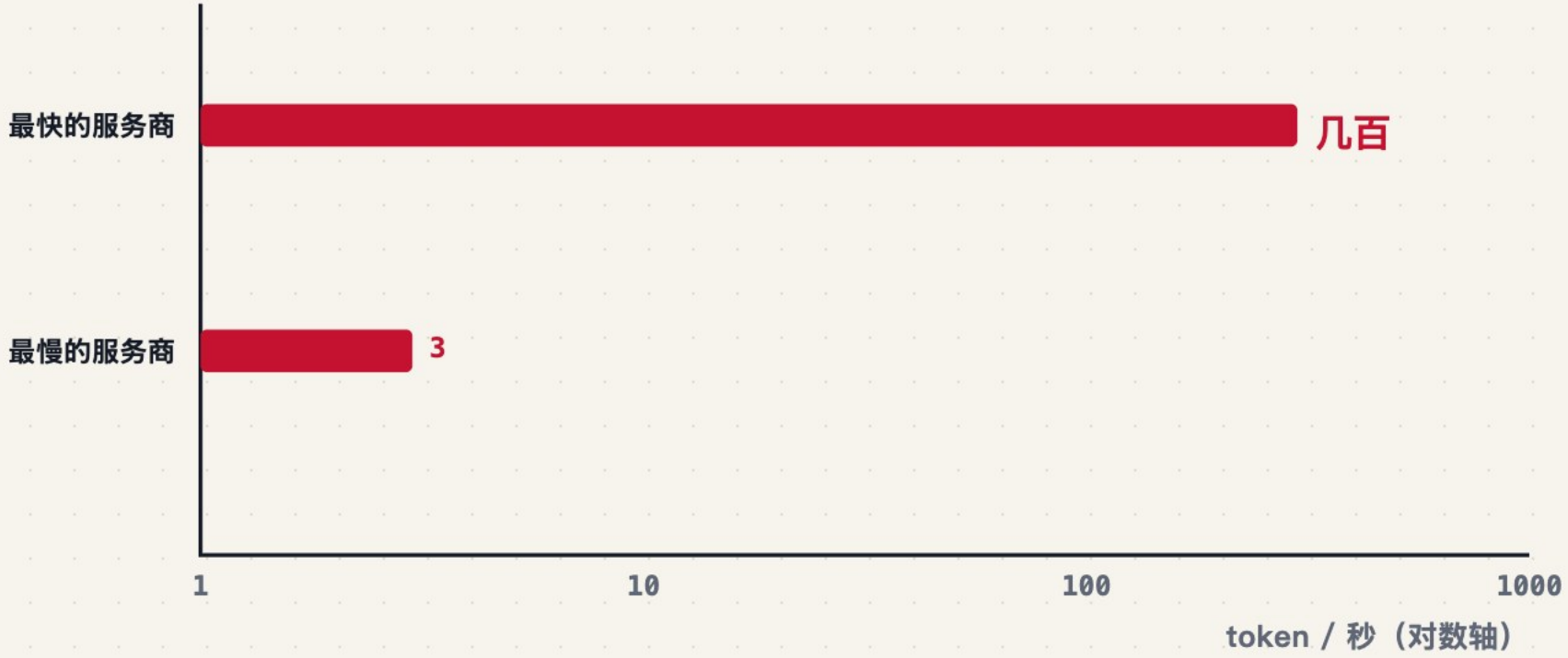
示意



课件 p.9（中文重绘）

同一个模型，换个服务商：速度差两个数量级

OpenRouter · GLM 5.2 · 不同服务商的生成速度



课上口头补充 · 条长示意



课件 p.9（中文重绘）

看哪个指标，取决于怎么用

和 agent 一起写代码



TTFT 要紧　TPOT 要紧

后台跑一整夜



~~速度几乎无所谓~~

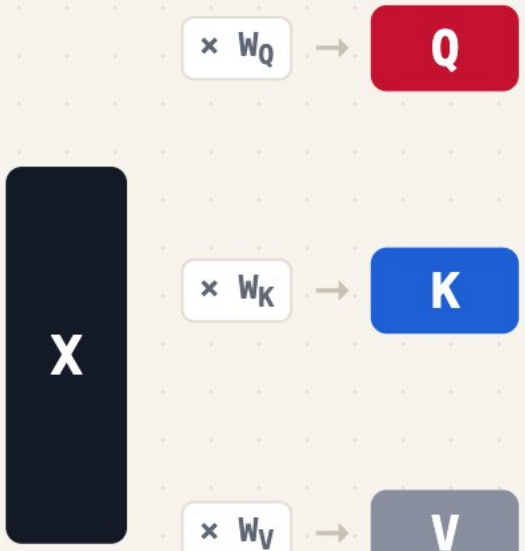


服务商先问：你更在乎哪个？

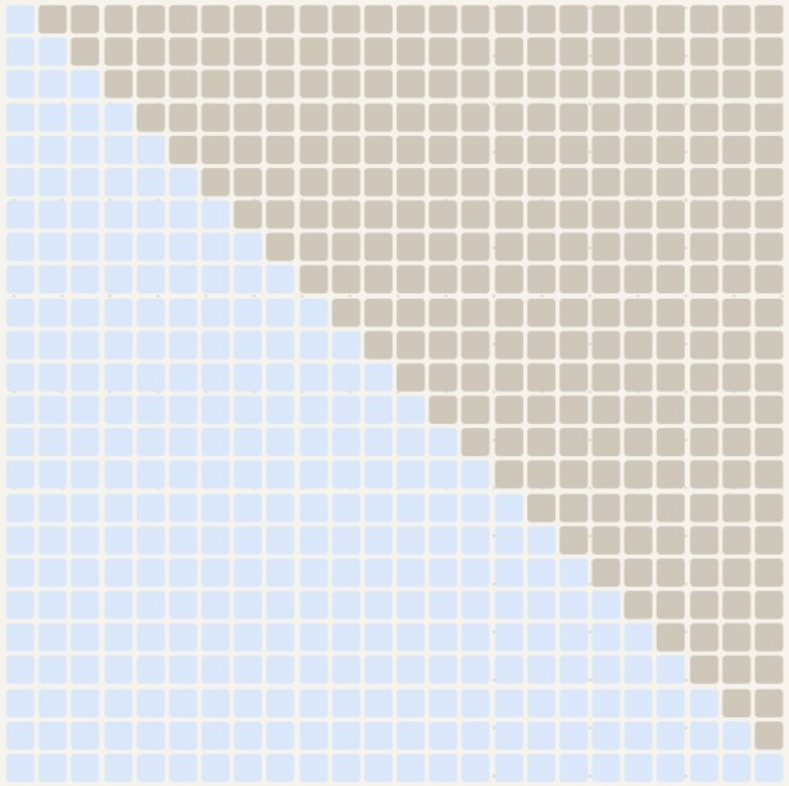


课件 p.9（中文重绘）

它对序列长度是平方的



这一讲后面：想办法绕开这个平方



点积 → softmax → 权重



权重加权 V → 输出

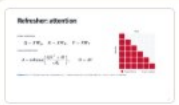
掩码：未来位置 = −∞

$$A = \text{softmax}((QK^T + M) / \sqrt{d_k})$$

$$O = AV$$

576

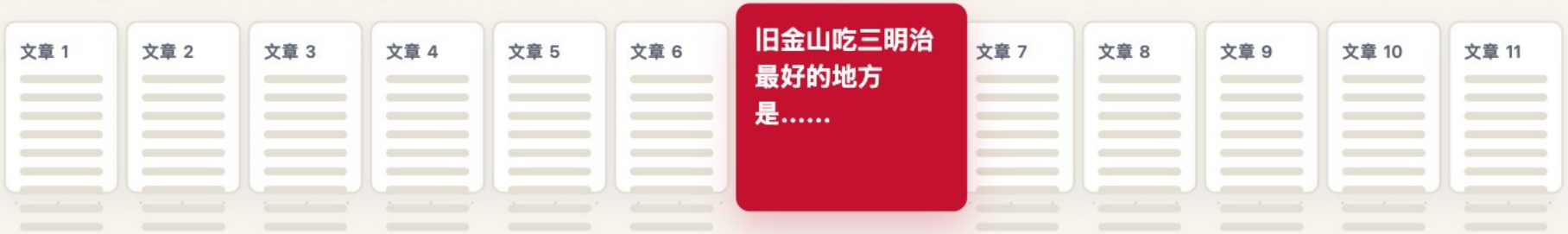
个格子 · 序列长度 24



课件 p.10（中文重绘）

号称的上下文 ≠ 真正能用的上下文

大海捞针：一堆 Paul Graham 的文章拼成一长条

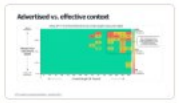


问：旧金山吃三明治最好的地方是?

模型

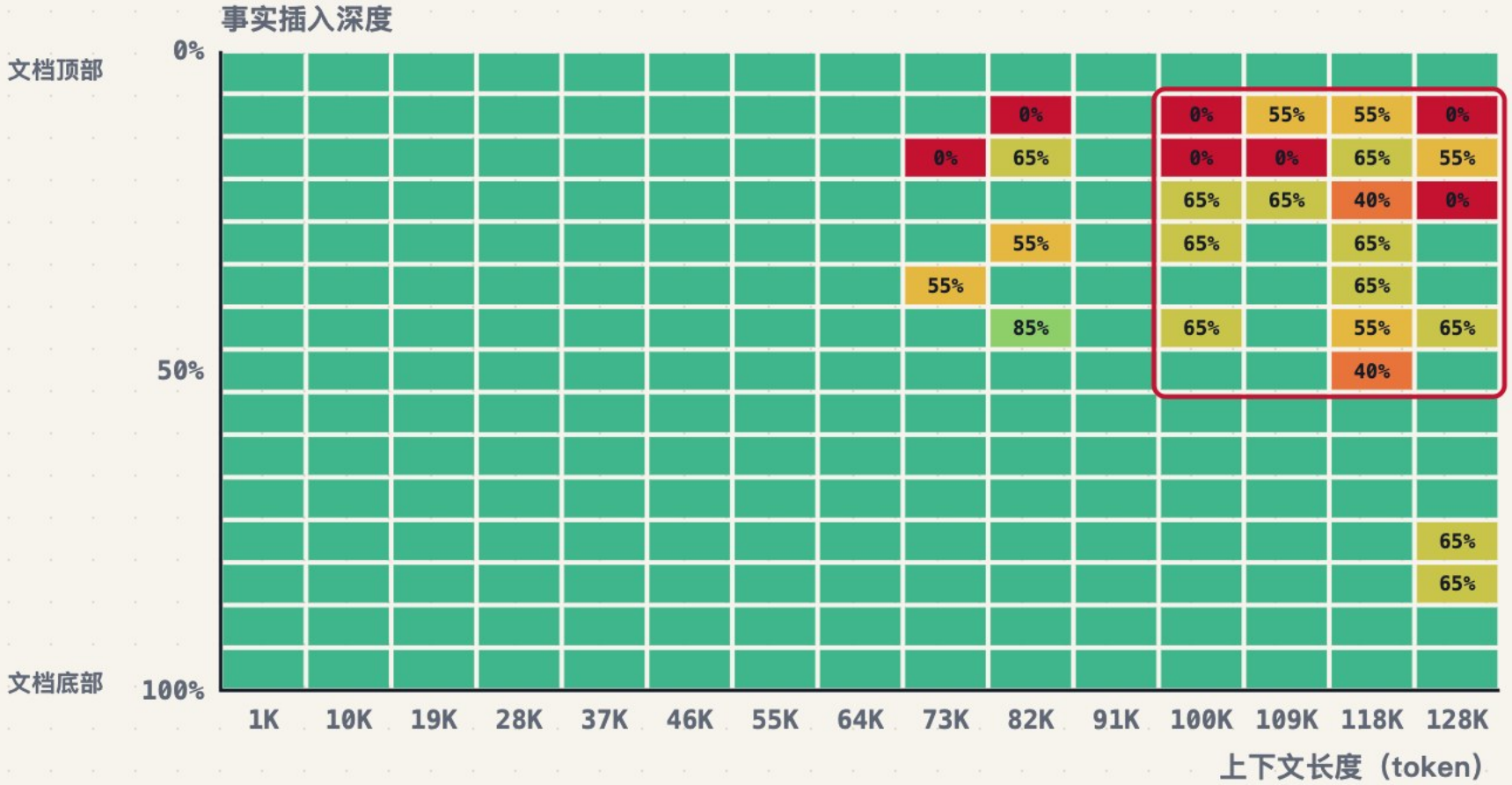
× 较弱的模型答错

上下文越拉越长 →



Kamradt 2023 · 课件 p.11 · 按原图读数

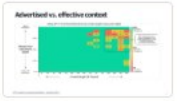
上下文很长时，用好它的能力可能变差



100K 以上

事实藏在 10%–50% 深度，准确率明显下降，有的 0%

RULER HELMET



Kamradt 2023 · 课件 p.11 · 按原图读数

agent 最常见的毛病：忘事

和 agent 的长对话（示意）

读取 src/app.py

`cat src/app.py → 412 行`

顺便把测试跑一下

`pytest → 3 failed`

修复 tests/test_io.py

`编辑 3 处 → 全部通过`

再清理一下没用的目录

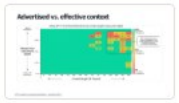
`rm -r notes/`

忘了前面的内容

说了别做，后来还是做了

用户的指令：一点一点、分好几次给

- 第 1 条：不要删 notes/
- 第 2 条：顺便跑测试
- 第 3 条：别改接口
- 第 4 条：先给我看 diff



课件 p.11 · 课堂讨论

全局注意力太贵：给比较次数设一个上限



10^{12} 次两两比较
一百万长度

$\approx 10^{20}$ 次运算
单条序列

课上口头补充

$$O(n^2) \longrightarrow O(wn)$$



局部：每个位置只比前 w = 3 个，共 102 次

w：不一定是窗口，是每一步比较次数的上限



课件 p.12–14（中文重绘）

混合架构：几次局部，一次全局

N × 局部计算 → 1 × 全局计算 → 重复



范围有上限 · 更新便宜

不局限在局部 · 信息传遍整个序列



课件 p.15–17（中文重绘）

六个模型：局部 : 全局 ≈ 3:1 到 5:1

模型	局部 / 有状态计算	全局计算	局部 : 全局（层数）
Qwen3.8-Flash-Next	Gated DeltaNet	QSA 稀疏检索	36 : 12 ≈ 3:1
GLM-5.3-Flash	Kimi Delta Attention	DSA + IndexPool	34 : 11 ≈ 3:1
Kimi K3	Kimi Delta Attention	稠密 Gated MLA	69 : 24 ≈ 3:1
Nemotron 3 Ultra	Mamba-2	稠密 GQA	48 : 12 = 4:1
Inkling-Small	512 token 窗口	稠密 GQA	35 : 7 = 5:1
DeepSeek-V4-Pro-0813	128 token 窗口分支	压缩稀疏 / 稠密	交错分支

DeltaNet 家族 = Gated DeltaNet、Kimi Delta Attention（KDA）



课件 p.15–17（中文重绘）

标准做法

几乎所有长上下文模型

某种局部机制 + 稀疏或稠密的全局注意力

1 **滑动窗口**



2 **线性注意力**



3 **稀疏注意力**



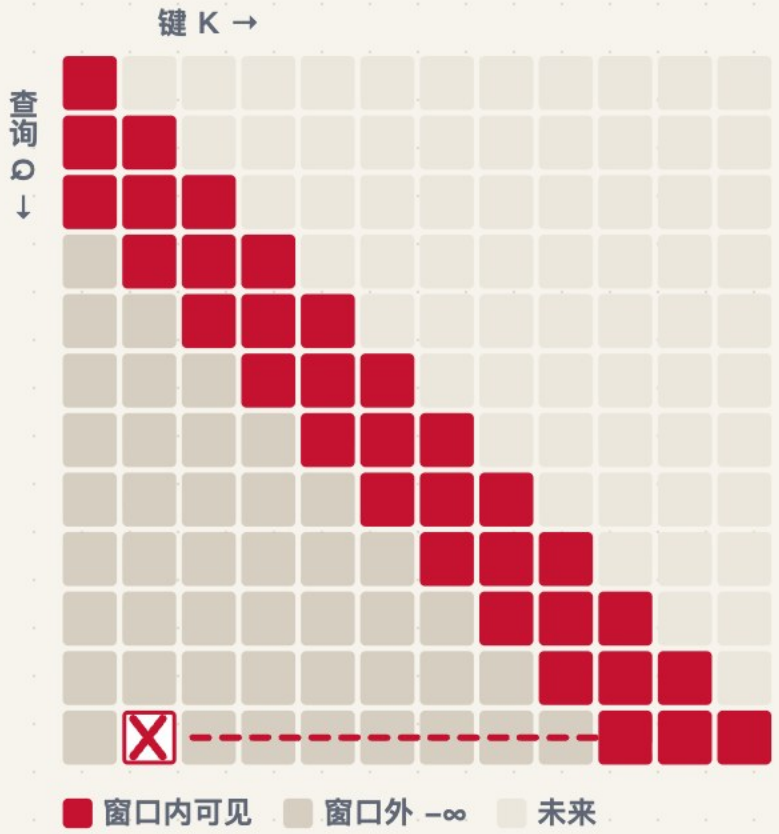
4 **KV 压缩**





课件 p.15–17（中文重绘）

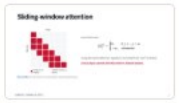
滑动窗口：公式不变，只换掩码



$$M_{ij}^{(w)} = \begin{cases} 0, & 0 \le i - j < w \\ -\infty, & \text{其他} \end{cases}$$

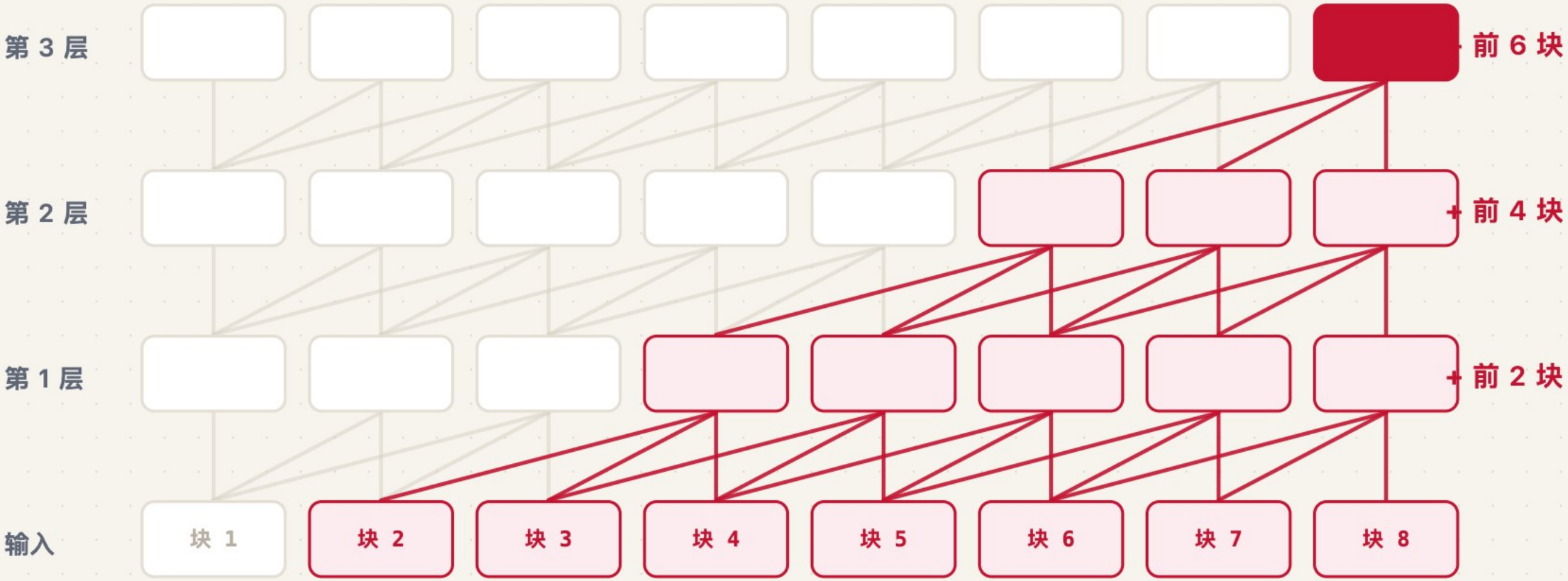
工作量 $O(n^2) \longrightarrow O(nw)$

× 单独一层：取不到远处的 token

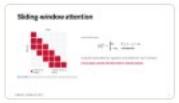


Longformer（Beltagy et al., 2020）· 课件 p.18

层数一多，能看到的范围一层层扩大



每块 512 token · 每层往前看两块（分块示意 · 课上口头补充）

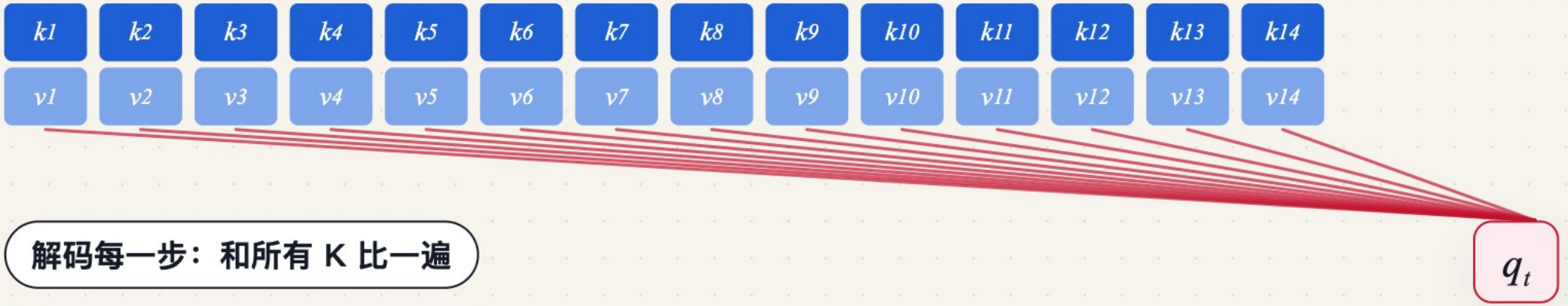


Longformer（Beltagy et al., 2020）· 课件 p.18

起点：softmax 注意力，KV 缓存不断变长

$$a_{ti} = \frac{\exp(q_t^\top k_i / \sqrt{d_k})}{\Sigma_j \exp(q_t^\top k_j / \sqrt{d_k})} \qquad o_t = \Sigma_i \, a_{ti} \, v_i$$

KV 缓存：每个位置的 K、V 都留着



解码每一步：和所有 K 比一遍

好处：能精确访问每一个 token



Transformers are RNNs（ICML 2020）· 课件 p.19–20

去掉 softmax：KV 缓存折叠成一个矩阵

1 $o_t = \Sigma_i \cancel{\text{softmax}_i(q_t^\top k_i / \sqrt{d_k})}\, v_i$

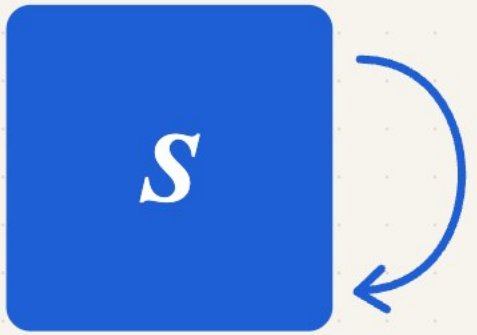
2 $o_t = \Sigma_i\, (q_t^\top k_i)\, v_i$ 点积直接当分数

3 $o_t = (\Sigma_i\, k_i v_i^\top)^\top\, q_t = S_t^\top q_t$ 按结合律重排

4 $S_t = S_{t-1} + k_t v_t^\top$ 每个新 token：常数时间更新

循环神经网络 · $O(n^2) \rightarrow O(n)$

之前所有的 K、V



固定大小 $d_k \times d_v$

一个从查询映射到值的线性函数

注意力做的也是这件事；不同的是，它可以递推



Transformers are RNNs（ICML 2020）· 课件 p.19–20

代价：丢了有所侧重的能力

softmax 权重

归一成加起来为 1，盯住相关的那几个



线性分数

示意

没有归一，平平的一排



只做加法写入

新内容一层层叠上去，
干扰不会被明确擦除



后面几种改进，都在做一件事

让状态矩阵 S，更好地预测
真正的注意力本来会给出的值



Transformers are RNNs（ICML 2020）· 课件 p.19–20

DeltaNet 家族：只写误差，再学会遗忘

Softmax | 线性 | DeltaNet | Gated DeltaNet | KDA

1 读 $\hat{v}_t = S_{t-1}^{\top} k_t$ 用当前状态预测

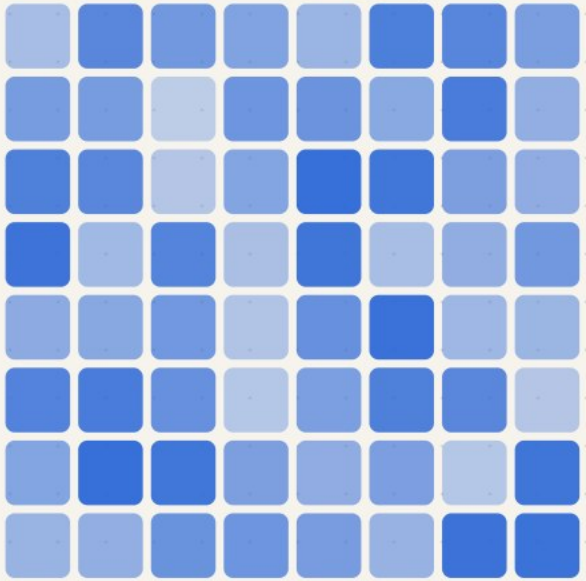
2 算误差 $e_t = v_t - \hat{v}_t$ 真值减预测

3 只写误差 $S_t = S_{t-1} + \beta_t k_t e_t^{\top}$



只沿 k_t 的方向修正 · 无关的记忆保持不变

状态矩阵 S（$d_k \times d_v$）



第 3 行：只有 k_t 方向被改写



DeltaNet / Gated DeltaNet / KDA · 课件 p.21–23

DeltaNet 家族：只写误差，再学会遗忘

Softmax | 线性 | DeltaNet | Gated DeltaNet | KDA

问题：之前积累的东西，没法淡化

很久以前一个很怪的 key，可能一直影响后面的预测

Gated DeltaNet：先整体衰减，再定向修正

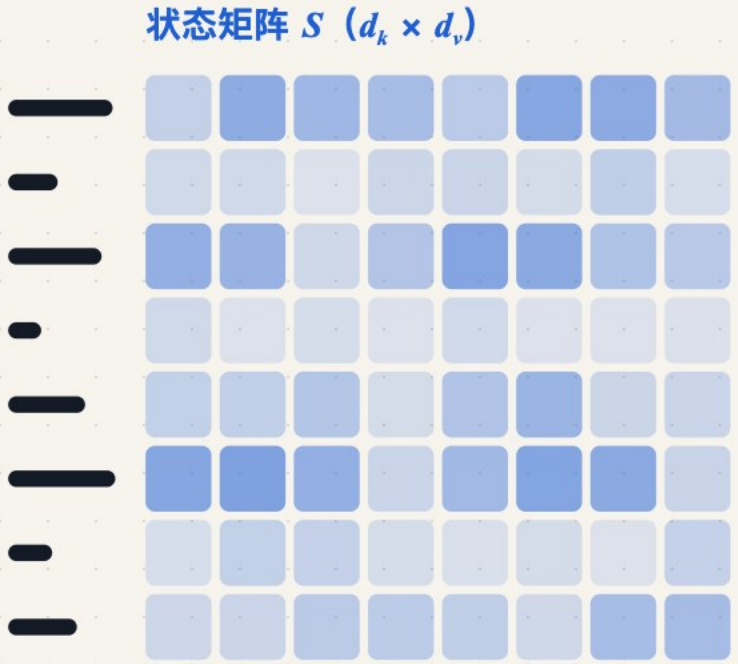
$\tilde{S}_{t-1} = \alpha_t S_{t-1}$ 衰减越强 → 忘得越快 → 越局部 · Qwen 在用

KDA：给 key 的每个通道一个寿命

$D_t = Diag(\alpha_t)$ 有的记很久，有的很快就忘 · GLM、Kimi 在用

Mamba：思路很相近

数学上更复杂，这里跳过





DeltaNet / Gated DeltaNet / KDA · 课件 p.21–23

稀疏注意力：只挑一部分看，而且限量



固定模式 与输入无关

局部窗口 + 一组固定的全局位置 G



✓ 成本可以预测

× 模式不随查询变化

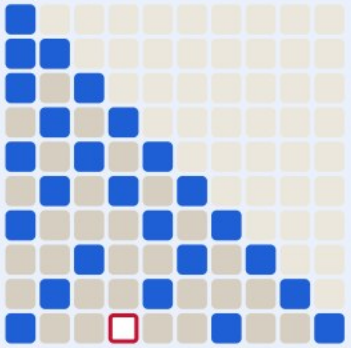
实现：一个 kernel 就够

约五年前 · OpenAI 课上口头补充

每隔 512 个 token 做一次 · 仍是平方的，但已经够用

按内容选 随查询变

投到很小的维度 → 粗算一遍 → 挑 top-K → 完整注意力



DeepSeek Sparse Attention

× 选择器一出错，
相关的 key 就被排除在外

实现：要两遍，可能更准

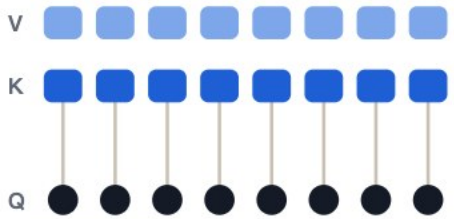


Longformer · DeepSeek Sparse Attention · 课件 p.24

效率可行，才谈得上一百万

MHA

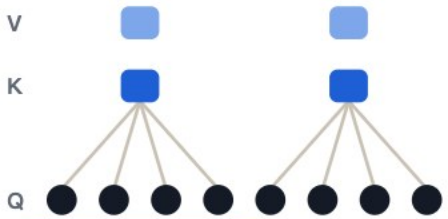
多头：每个头一组 K、V



缓存 8 组

GQA

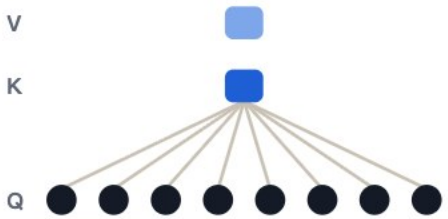
分组：几个查询头共用一组



实践中 ÷4 或 ÷8

MQA

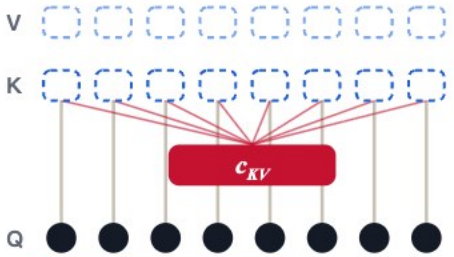
多查询：所有头只用一组



缓存 1 组

MLA

潜在压缩：存小向量，用时还原



只缓存小的潜在向量

÷4 或 ÷8：课上口头补充

一张 GPU 的显存　　能同时跑的请求：8　示意



▪ 模型权重　▪ 每个请求的 KV 缓存

GQA　减的是头的数量

MLA　减的是每个头的大小

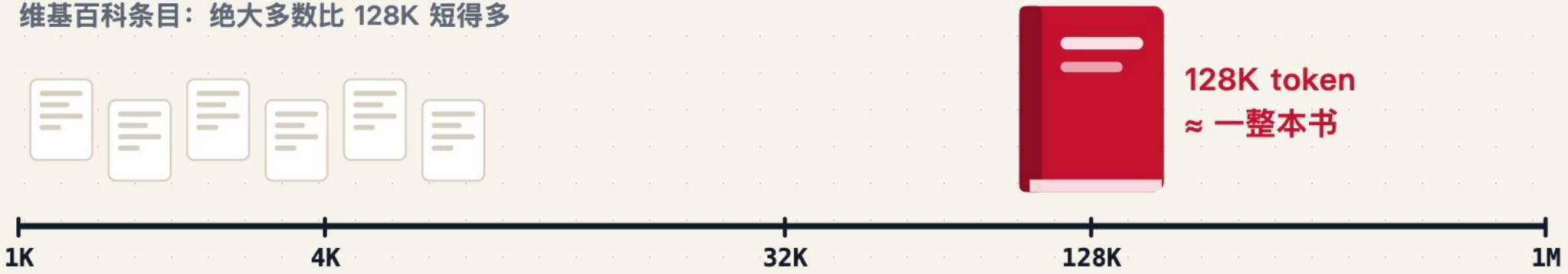
＋ 两者可以组合使用



GQA（Ainslie et al., 2023）· MLA（DeepSeek-V2, 2024）· 课件 p.25

训练：难点在数据

维基百科条目：绝大多数比 128K 短得多



网上连贯的长内容不多，书也没有那么多



课件 p.26–27（中文重绘）

由短到长：三步把长度拉上去

短预训练

4K–8K

数据多 · 成本低

继续训练

32K–128K

长文档 · 拼接序列

长适配

128K+

任务轨迹 · 合成课程

一条拼接序列：十篇文档



✓ 对效率有用

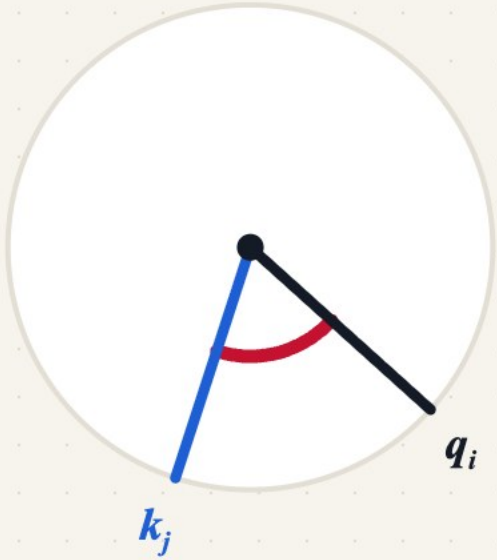
× 学不到跨文档 （本来也不该跨）



课件 p.26–27（中文重绘）

位置编码：从加一个向量到旋转 Q、K

词向量 $E[x_t]$ ＋ 位置向量 p_t ＝ 第一层输入 z_t

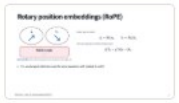


RoPE：按位置把 Q、K 旋转

$$\tilde{q}_i = R(i)\, q_i, \qquad \tilde{k}_j = R(j)\, k_j$$
$$\tilde{q}_i^{\top} \tilde{k}_j = q_i^{\top} R(j-i) k_j$$

`i = 6, j = 9 → j − i = 3`

绝对位置不影响该不该互相注意 · 用得非常多



RoFormer · NoPE（NeurIPS 2023）· 课件 p.28–30

NoPE：根本不要位置编码

词向量 $E[x_t]$ + ~~位置向量 p_t~~ = 第一层输入 z_t　NoPE

> 常听说：不加位置编码，不同位置上的同一个“猫”，表示完全一样？

答案：因果掩码



位置 2 的“猫”：看到前 1 个



≠

位置 5 的“猫”：看到前 4 个



从第一层注意力开始，两者就不一样了

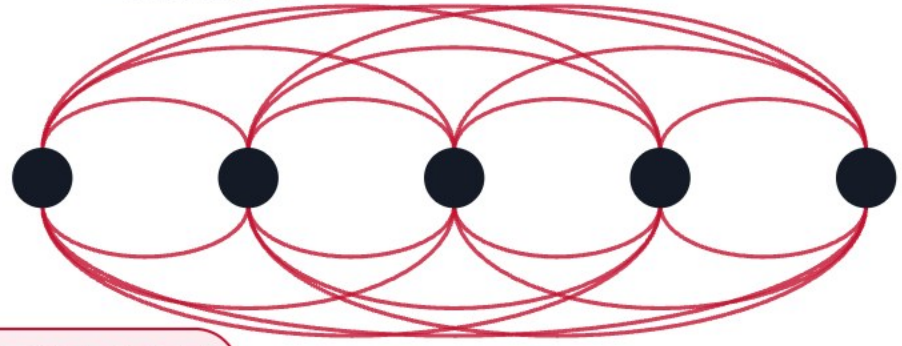


RoFormer · NoPE（NeurIPS 2023）· 课件 p.28–30

自回归模型，本来就知道顺序

早年的编码器

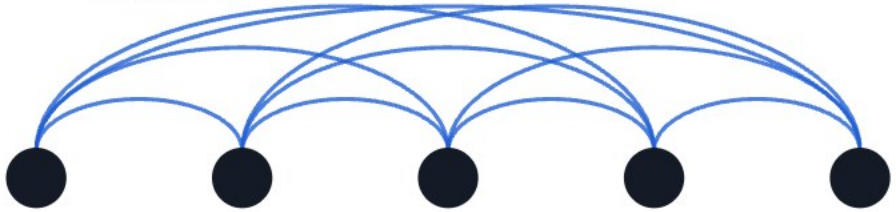
所有 token 互相都能看到



需要位置编码

自回归 Transformer

每个 token 只看前面



不需要

再加上 Gated DeltaNet 这类带衰减的局部机制

衰减本身就带着位置偏置：离得越远，留下的越少

远 → 近



RoFormer · NoPE（NeurIPS 2023） · 课件 p.28–30

想扩到好几倍长：两个问题



① 没见过

距离超过训练长度的两个 token，
模型从没见过它们互相注意

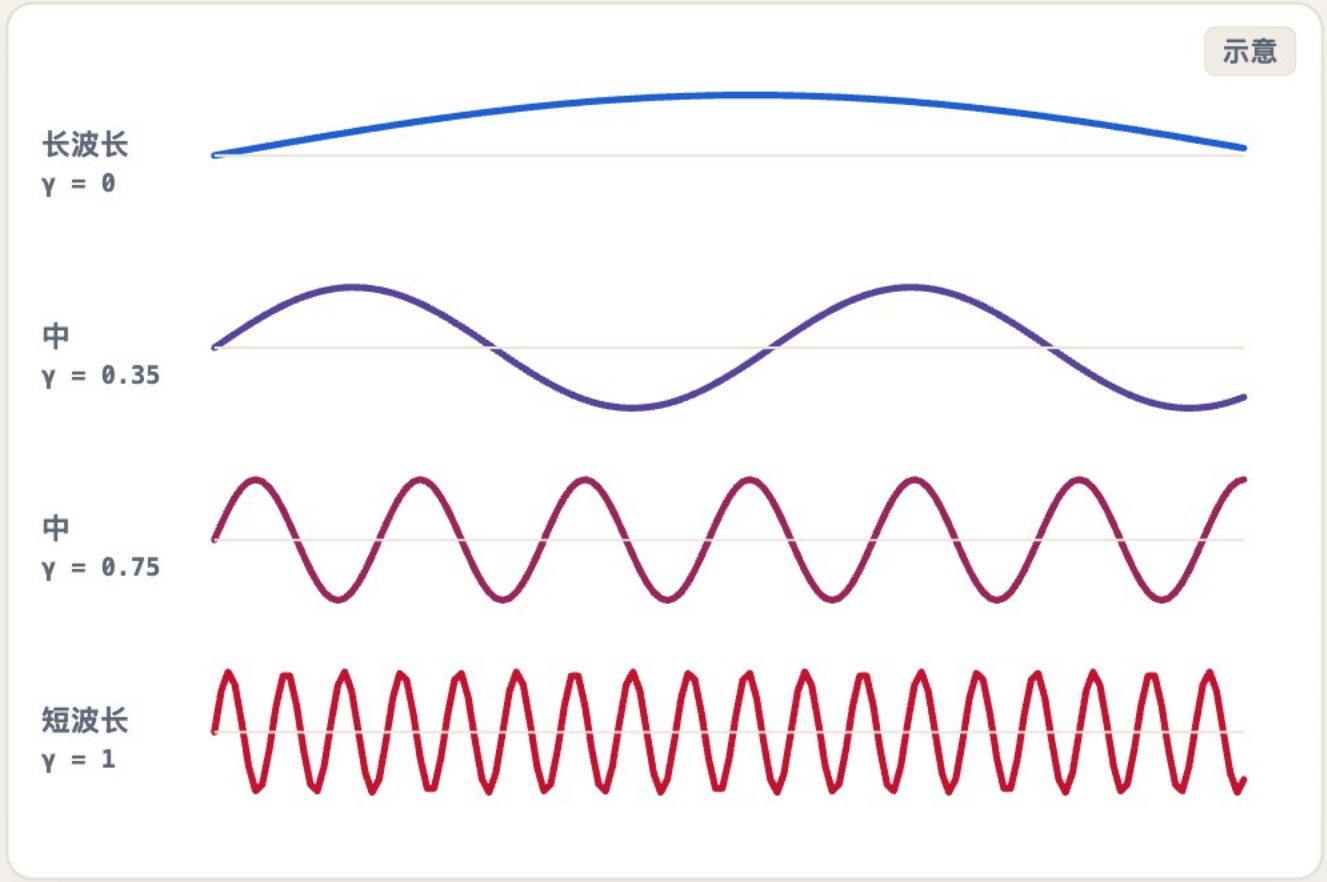
② 不只是 RoPE

自回归掩码这些因素也有影响，
不能指望直接好用



Position Interpolation（2023）· YaRN（ICLR 2024）· 课件 p.31–32

YaRN：按频率区别对待



位置插值

0 … L

$$R(t) \longrightarrow R(t/s)$$

所有维度按同一个比例 s 拉伸

YaRN

$$\theta'_t = (1-\gamma_t)\,\theta_t/s + \gamma_t\theta_t$$

长波长完全插值，短波长保留局部相位，
中间平滑过渡 · 像 KDA：有的管远，有的管近

注意力温度修正

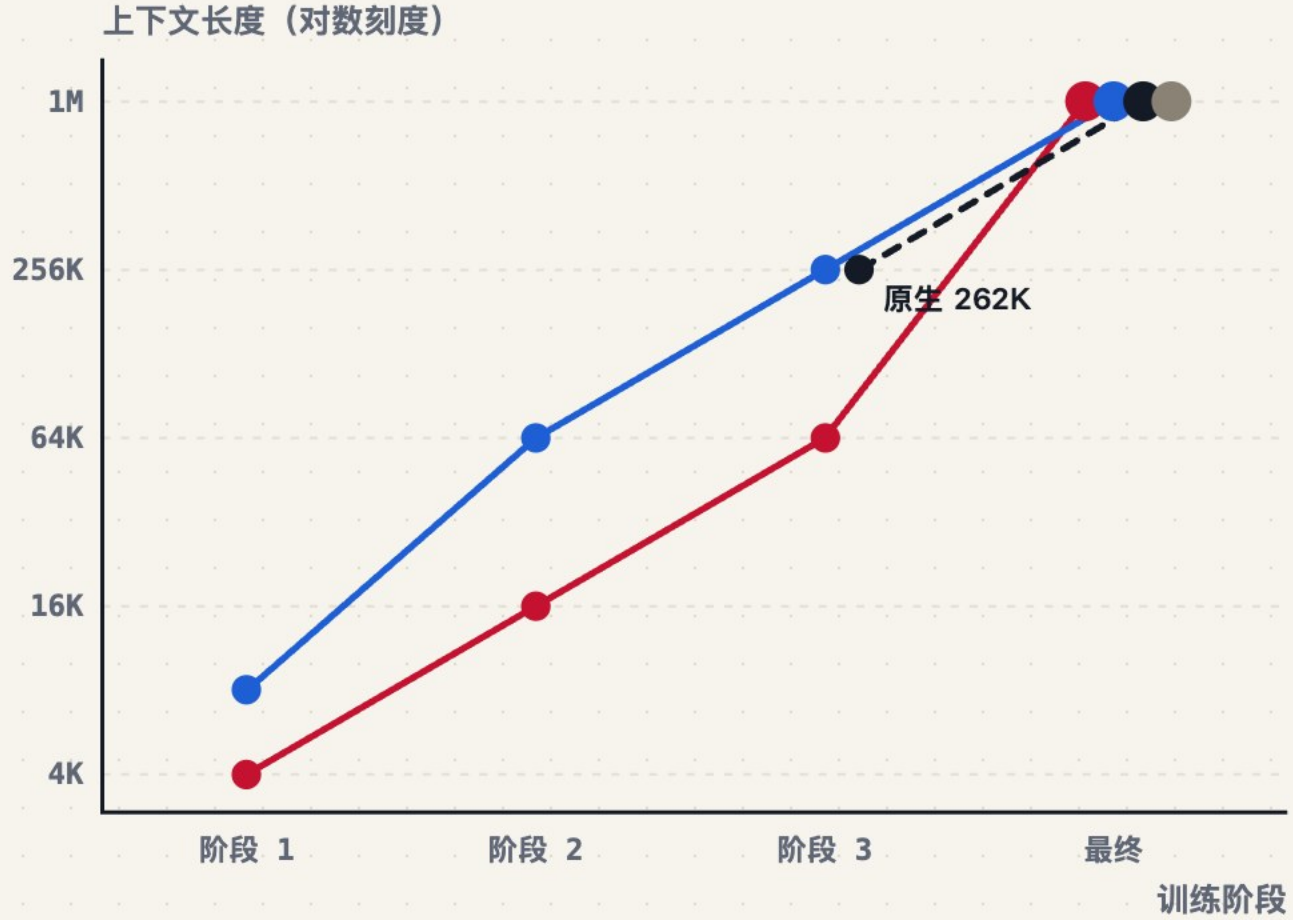
$$\sqrt{(1/\tau)} = 0.1 \ln s + 1$$

在 LLaMA 上拟合 · 是配方，不是常数



Position Interpolation（2023）· YaRN（ICLR 2024）· 课件 p.31–32

大致一半用 RoPE，一半不用



DeepSeek V4 部分 RoPE
4K → 16K → 64K → 1M，再加 YaRN

Kimi K3 NoPE
8K → 64K → 256K → 1M

Qwen3.8-Flash-Next 部分 RoPE
原生 262K，用 YaRN 扩到 1M

GLM-5.3-Flash NoPE
原生 1M，不用缩放 RoPE

课件另列：Nemotron 3 Ultra（Mamba 隐式顺序）、Inkling（相对位置表示）

数据：课件 p.33（各家技术报告）



课件 p.33 · 各家技术报告

长数据从哪来：四类来源

1

长文档

连贯的自然结构

书　代码库　多篇文档　专利

书、代码库、同一主题的多篇文档
· 代码库通常是连贯的

2

拼接样本

token 利用率高



要注意边界，和样本之间的串扰

3

agent 轨迹

动作 + 观察



教长程恢复、状态跟踪和工具使用
· 跑一会儿就非常长

4

合成任务

依赖关系可控



证据和干扰项放在设计好的位置

代码库、专利、agent 轨迹的说法：课上口头补充

连贯的长来源 + 答案依赖远处证据的任务 + 逐步增长的长度

闭源　架构 ✕ 连架构都不公开

开源　架构 ✓　数据配比和训练策略 ✕



课件 p.34–35（中文重绘）

上下文并行：序列切块，K、V 绕环传



和计算重叠的通信

每块算自己的注意力，

同时把 K、V 发给下一台

总工作量仍是 $O(n^2 d_k)$

精确计算，不是稀疏近似

上下文超过约 32K：很重要 课上口头补充

反向传播 kernel

标准注意力：上下文并行 kernel ✓

Gated DeltaNet：

上下文并行 + 反向传播 kernel ?

网上找不到现成的

只能自己写 · 实践中常遇到



Ring Attention（Liu et al., 2023）· 课件 p.36

一次次调用之间，复用 KV

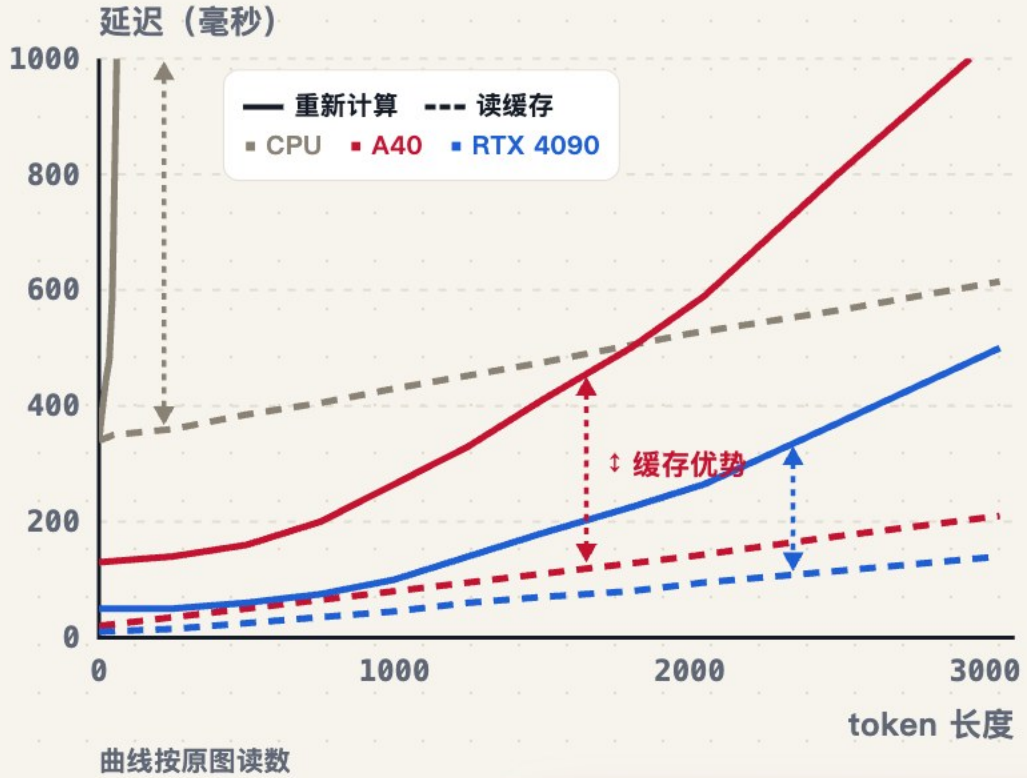




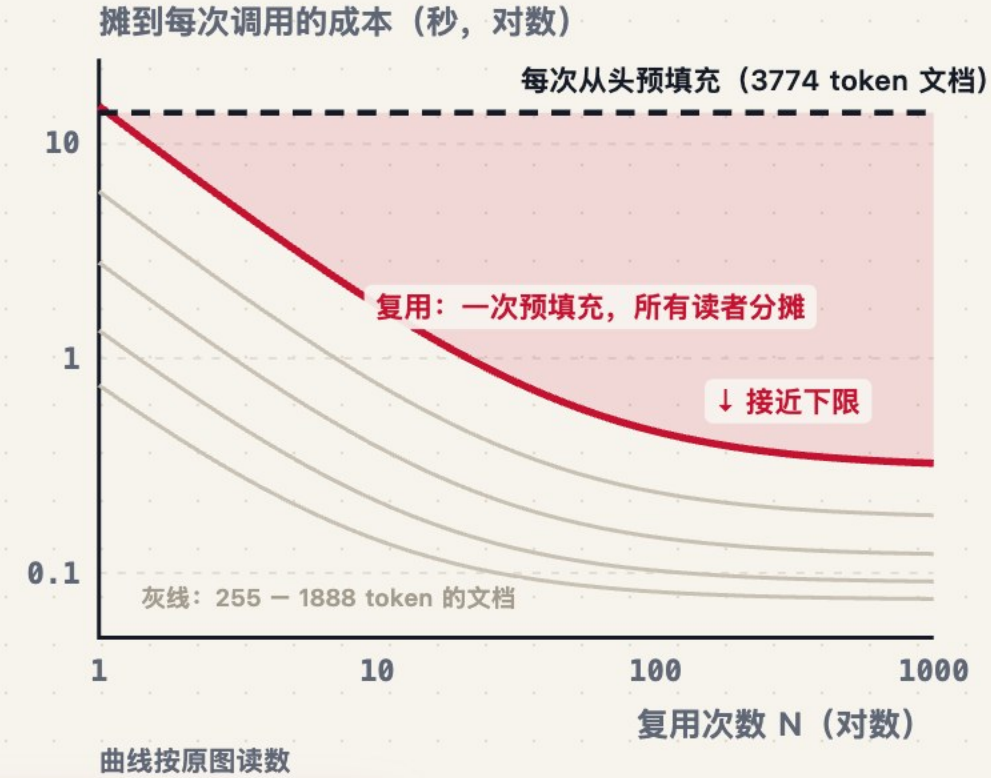
Prompt Cache（MLSys 2024）· 课件 p.37–38

复用越多，每次调用越接近下限

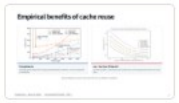
① 重新计算 vs 读缓存 Prompt Cache · Gim et al., 2024



② 同一份文档被多少个 agent 读 Can I Buy Your KV Cache? · 2026



实测的物理成本 ≠ 服务商的收费



Prompt Cache（2024）· Can I Buy Your KV Cache?（2026）· 课件 p.39

缓存读取，一般只收十分之一

模型	缓存读取	正常输入	输出 正常输入 = 1 · 2026 年 8 月 30 日核对
DeepSeek V4 Flash	≈1/30	1	3×
GLM-5.2	≈1/5	1	≈3×
Kimi K3	1/10	1	5×
GPT-5.6 Luna	1/10	1	6×
GPT-5.6 Sol	1/10	1	5×
Claude Opus 5	1/10	1	5×

缓存 = 命中读取，写入缓存可能另计 · DeepSeek 为闲时到忙时的区间（两端比例相同） · 条形长度按标价换算

没缓存上 同一段前缀按正常输入价付，大约是缓存价的 10 倍

Claude Opus 5 的三种价



为什么便宜：查出来就行

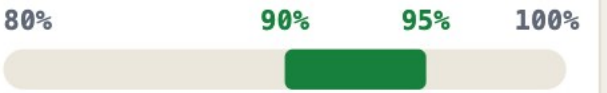
课上口头补充

重新算 注意力 → 前馈网络（贵）

读缓存 查出 K、V → 跳过

缓存命中率多高算好

课上的经验值



90%–95%：一切正常

更低：也许没事，也可能哪里漏了



课件 p.40（各家官网标价，换算成比例）

两个开源推理框架，都起源于 KV 缓存管理

vLLM

PagedAttention

按页管理 GPU 显存

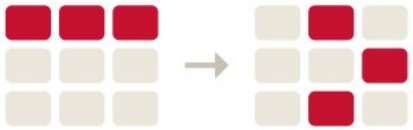
SGLang

RadixAttention

把缓存过的前缀组织成树

操作系统

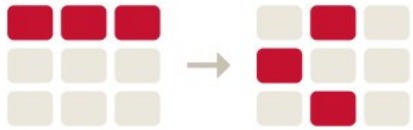
虚拟内存页 → 物理内存页，靠页表对应



≈

GPU 显存里的 KV 缓存

逻辑 KV 块 → 物理 KV 块，靠块表对应





vLLM / PagedAttention（Kwon et al., SOSP 2023）· 课件 p.41

PagedAttention：块表把逻辑块对到物理块

请求 A　提示词 *“Four score and seven years ago our”* → 输出 *fathers* → *brought* → …

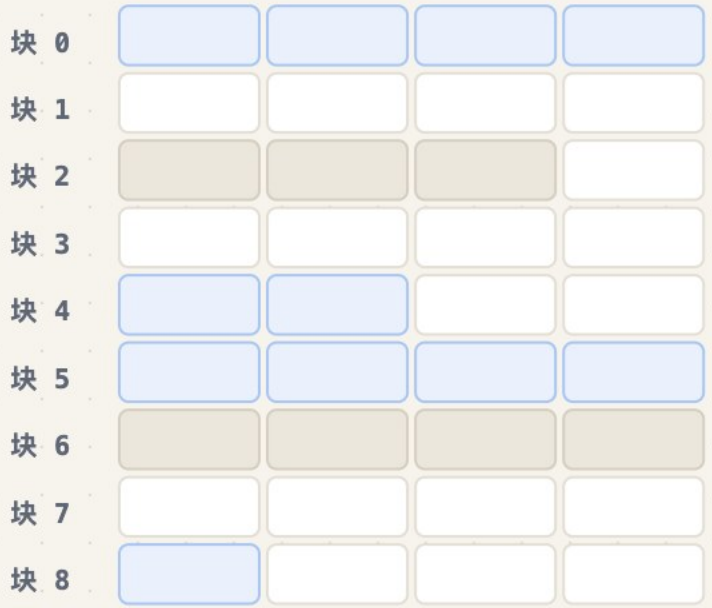
逻辑 KV 块

块 0	*Four*	*score*	*and*	*seven*
块 1	*years*	*ago*	*our*	*fathers*
块 2	*brought*			
块 3				

块表

物理块号	已填
7	4
1	3 → 4
3	1

物理 KV 块 GPU 显存



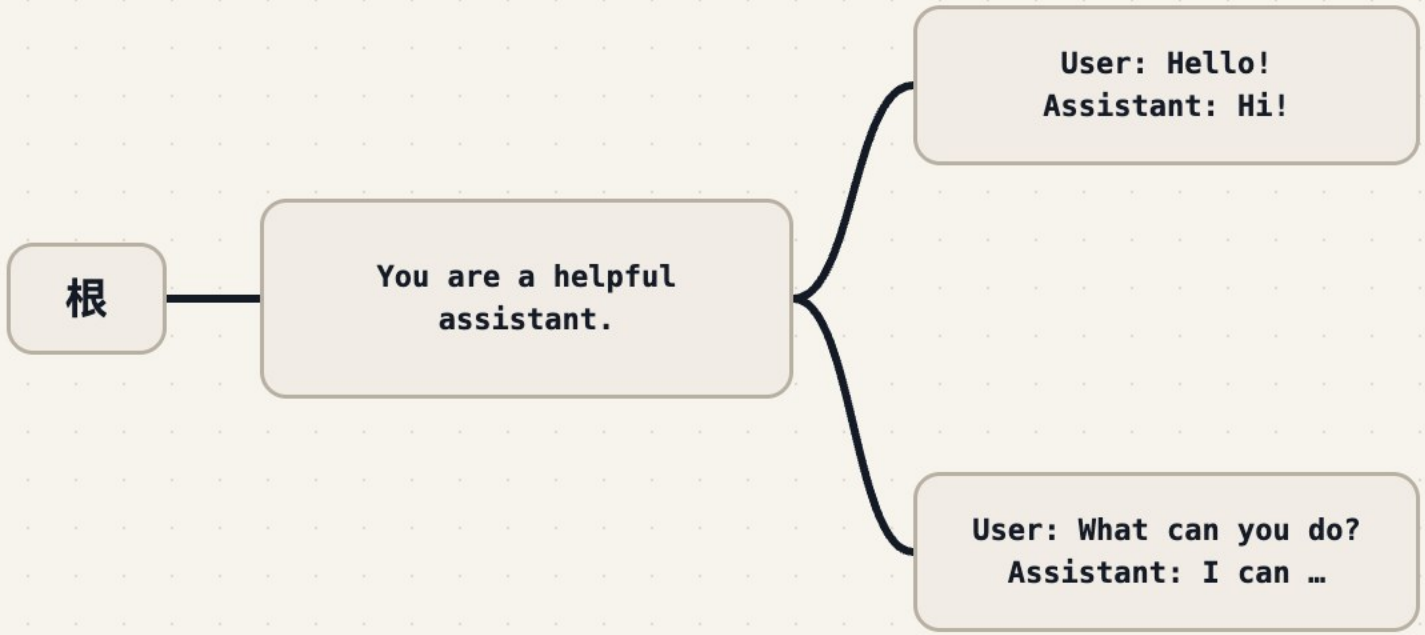
逻辑上连续，物理上分散　块表逐块对应

同时生成很多条序列　一条生成完，整块释放



vLLM / PagedAttention（Kwon et al., SOSP 2023）· 课件 p.41

RadixAttention：缓存过的前缀，组织成一棵树



① 匹配前缀　② 分叉共享　③ 淘汰叶子

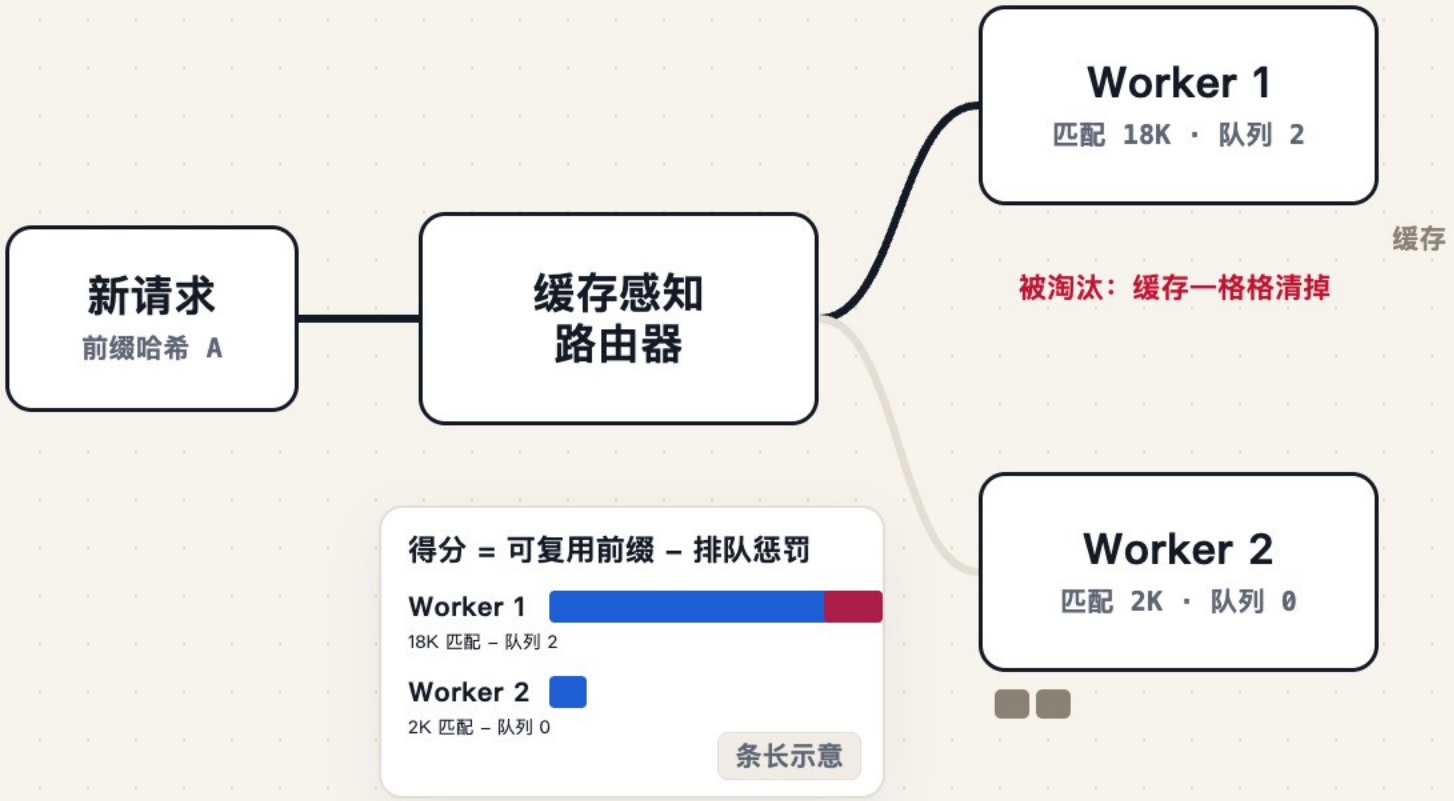
对 agent 重要得多　　对话更长 · 请求更频繁

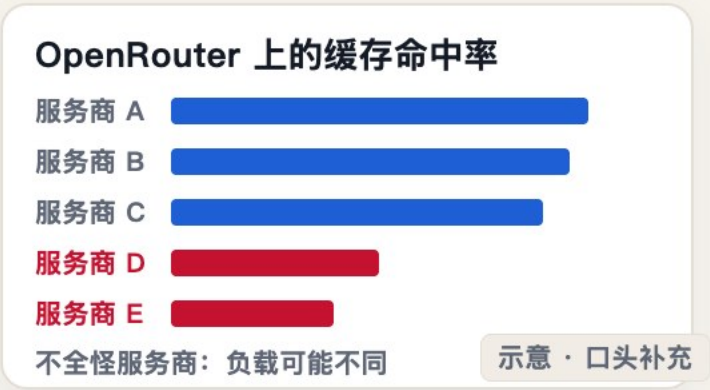
最早：聊天机器人　→　现在：agent



SGLang / RadixAttention（Zheng et al., 2023）· 课件 p.42

缓存感知路由：送到缓存最匹配的那台











SGLang v0.4 缓存感知负载均衡 · 课件 p.43

写 harness：千万别无意中打破缓存

稳定前缀 指令 · 工具 · 示例

往后追加 用户轮次 · 观察结果 14:03:38

真需要时间：放进最后一条用户消息或工具结果里 ✓

1

稳定前缀

指令 · 工具 · 示例
放在最前面，保持不变

2

变化的状态往后追加

新的用户轮次
观察结果

3

避免不必要的扰动

时间戳
换了顺序的 schema

4

量出复用

缓存 token 数 · TTFT
淘汰



课件 p.44（中文重绘）

写 harness：千万别无意中打破缓存

稳定前缀 指令 · 工具 · 示例

往后追加 用户轮次 · 观察结果

1

稳定前缀

指令 · 工具 · 示例
放在最前面，保持不变

2

变化的状态往后追加

新的用户轮次
观察结果

3

避免不必要的扰动

时间戳
换了顺序的 schema

4

量出复用

缓存 token 数 · TTFT
淘汰

贵模型

× 10

按正常价重新付一遍前缀

难的一步交给它

第 1 步 贵 | 第 2 步 便宜 | 第 3 步 便宜 | 第 4 步 贵

切换模型省钱：次数要省着用

便宜模型

前缀已缓存

后面几步交给它省钱



课件 p.44（中文重绘）

写 harness：千万别无意中打破缓存

稳定前缀 指令 · 工具 · 示例

往后追加 用户轮次 · 观察结果

1

稳定前缀

指令 · 工具 · 示例
放在最前面，保持不变

2

变化的状态往后追加

新的用户轮次
观察结果

3

避免不必要的扰动

时间戳
换了顺序的 schema

4

量出复用

缓存 token 数 · TTFT
淘汰

前缀稳定 · 只往后加 · 不乱动前缀 · 量出命中



课件 p.44（中文重绘）

一个压缩的例子

压缩前 五步历史

观察 CI 任务 30 分钟后超时

动作 查看任务日志

观察 24K 行日志 · 反复出现“address already in use”

动作 设 `--workers=1` 重跑

观察 测试通过 · 日志存在 `/tmp/ci.log`



压缩



课件 p.45–46（中文重绘）

一个压缩的例子

压缩前 五步历史

压缩

压缩后 一段工作状态

目标	修复 CI 超时
原因	端口冲突
已验证	`--workers=1` 能通过
证据	`/tmp/ci.log`
下一步	用测过的 worker 设置去开 PR



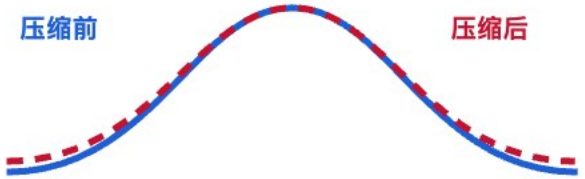
课件 p.45–46（中文重绘）

把压缩看成状态估计

完整历史 H_t → 压缩器 C_B（在 token 预算 B 内）→ 工作状态 $\hat{s}_t$（够用就行）

1 行为保真

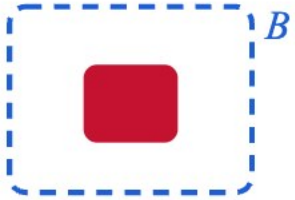
曲线示意



$$p(A_{未来} \mid H_t) \approx p(A_{未来} \mid \hat{s}_t)$$

压缩后 agent 接下来的动作，和压缩前一样或相近，甚至更好

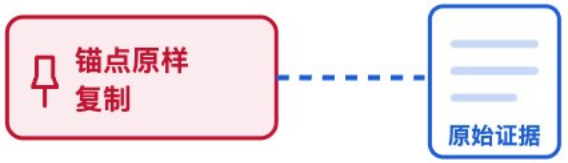
2 有界



$$|\hat{s}_t| \le B$$

状态要足够小，小到能放心接着干活

3 可恢复



留一根指针

关键锚点原样复制，保留指向原始证据的指针



课件 p.47（中文重绘）

压缩时，什么能留下来

好的 harness 刻意不删开头

模型能看到的上下文

会话开头

锚点	目标 · 约束	原样保留
检查点	决策 · 进度 · 各种 ID	编码进去
最近的尾部	正在进行的尝试 · 刚拿到的结果	原样保留
指针	→ 证据库 · 磁盘上的完整历史	只留指针

证据库 放到上下文外面

- 3 条失败
- 1 条命令
- 1 个文件路径

24K 行日志

丢弃

参考：MemGPT · LongLLMLingua
会话开头、磁盘历史：课上口头补充



课件 p.48（中文重绘）

压缩时，什么能留下来

好的 harness 刻意不删开头

模型能看到的上下文

会话开头

锚点	目标 · 约束	原样保留
检查点	决策 · 进度 · 各种 ID	编码进去
最近的尾部	正在进行的尝试 · 刚拿到的结果	原样保留
指针	→ 证据库 · 磁盘上的完整历史	只留指针

证据库 放到上下文外面

- 3 条失败
- 1 条命令
- 1 个文件路径

24K 行日志

丢弃

完整历史

命令行编码 agent：通常存在磁盘上

参考：MemGPT · LongLLMLingua

会话开头、磁盘历史：课上口头补充



课件 p.48（中文重绘）

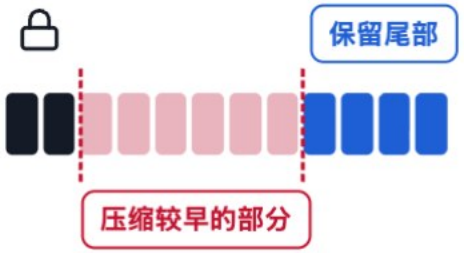
落到实现上：四个决定

1 触发



- 超过 token 阈值
- 服务商报溢出
- 用户手动要求
- 给下一次输出留出空间

2 选择



- 保护开头
- 压缩较早的部分
- 保留最近的尾部
- 切点落在完整的轮次 / 工具调用边界

3 替换



- 服务商给的不透明对象：加密、读不了
- 据说只有 OpenAI 这么做，免得压缩算法被学走（课上口头，未核实）

4 恢复



- 原始历史保持不变
- 能重试或重置
- 保留可搜索的历史
- 规定好压缩失败时怎么办

参考实现：Codex · OpenCode · Pi · Hermes · OpenHands　　200K 为课上口头举例



课件 p.49（中文重绘）

反复压缩会漂移

原始历史

用 CUDA 12.4



课件 p.50（中文重绘）

反复压缩会漂移

原始历史 → 摘要 1 → 摘要 2 → 摘要 3

约束没了



课件 p.50（中文重绘）

压得越多，丢得越多 → agent 跑偏

1. 锚点 **原样** 复制
2. 保留最近的尾部
3. 回原始证据检索

3 真塞不下了，agent 怎么压缩历史?

摘要会丢什么 → 这里 （反复压缩会漂移）
检索怎么补 → 上一段 （按需检索）



课件 p.50（中文重绘）

评测压缩：看压完能不能接着干

1 埋入状态

约束 · 决策 · 产物

→

2 压缩

用线上实际的策略

→

3 继续

衡量之后的决策

状态回忆

约束和标识符还在不在

任务成功

接下来做得对不对

效率

token · 延迟 · 成本

稳定性

反复压缩之后还行不行

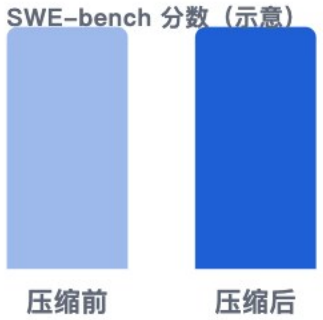


课件 p.51（中文重绘）

为什么不直接跑基准?

实践中常见

评测里 花一个月做出的压缩策略



分数一点没掉

token ↓ 省了很多

上线后 每压缩一次就提一个 PR

- PR 同一个功能
- PR 同一个功能（又一个）
- PR 同一个功能（又一个）
- PR 同一个功能（又一个）
- PR 同一个功能（又一个）

5 个 PR

真实使用 多轮，中途有补充（示意）

用户：修好 CI 超时

agent：在查日志……



压缩

评测集 单轮任务

用户：任务描述（只有这一条）

没有"后来的用户消息"
→ 测不出它们被压掉



课件 p.51（中文重绘）

光跑基准不够

除非基准真能代表每一种用法，
还得在真实使用中测。



课件 p.51（中文重绘）

长上下文，是一个分层的系统

5	压缩	保住意图、任务状态、最近的工作和恢复路径，用"能不能接着干"来评测	CUDA 12.4
4	缓存	稳定的前缀省掉重复预填充，但不减少模型看到的历史	
3	架构	局部、循环、稀疏、压缩：决定哪些历史还能访问到	
2	推理	预填充、解码、注意力和 KV 显存，各有各的瓶颈	
1	增长	循环一遍遍读入指令、动作和观察，总输入按平方涨	